Homework 3: Microarchitecture Experiments

Track B — Skylark and Artemisia

pgpatel

ECE 492/592 — Fall 2026

Exercise	Selected experiment	Available points
1	Evidence of pipelined execution	10
2	Evidence of out-of-order execution	10
3	Evidence of branch prediction	10
4	Reorder buffer (ROB) size	30
5	Physical register file (PRF) size and register class	35
14	Effective instruction delivery / fetch estimate	10
	Total selected	**105**

Scope. This report compares the supplied ordinary-execution measurements on two course machines. No enclave experiments were performed. Cache and prefetcher reverse engineering, including DMPs, are outside the scope. The memory accesses in the ROB/PRF probes are a way to delay retirement, not an investigation of cache or prefetcher design.

Evidence status. The ROB/PRF sweeps retain repeated process measurements. The pipeline, scheduling, branch, and instruction-delivery CSVs retain only one timing per condition. Repeating instructions within a timed loop amortizes overhead but does not provide independent timing samples. Consequently, the repeated-measurement requirement is not yet satisfied for those four exercises; no error bars or confidence intervals are invented. Selecting 105 available points does not establish that every exercise is fully demonstrated.

AI disclosure. OpenAI Codex assisted with reviewing the supplied code and data, producing plots, and drafting this LaTeX report. No new machine measurements were generated during report preparation. The author must review the analysis and be able to explain the submitted experiments. A separate AI-use log accompanies this project.

1 Machines, provenance, and measurement conventions

	Skylark	Artemisia
Host	`skylark.ece.ncsu.edu`	`artemisia.ece.ncsu.edu`
CPU model	`AMD EPYC 7532 32 Core`	Intel Xeon Gold 5420+
Microarchitecture / family	`Zen 2(Rome), Family 23h, Model 49`	Sapphire Rapids (published identification)
Configuration record	Snapshot dated November 04, 2026	Snapshot dated October 30, 2026

Artemisia's CPU model comes from `system_configuration_snapshot/snapshot.csv`. Intel identifies this processor as Sapphire Rapids [2]. The snapshot records SMT enabled, isolated CPUs, and turbo disabled through the Intel P-state setting. These historical settings do not establish the conditions of every benchmark run. Skylark's CPU model cannot be inferred reliably from a timing curve or its hostname.

File mapping. The author confirmed that the four simple experiments' unsuffixed CSV files belong to Artemisia, while files ending in `_skylark.csv` belong to Skylark. In the repeated sweeps, `prf_mymachine_*` denotes Artemisia and `prf_skylark.ece.ncsu.edu_*` denotes Skylark. The report package preserves these original names, adding a folder prefix to the simple CSVs.

Timing. The first three experiments use `CLOCK_MONOTONIC` elapsed seconds. The instruction-delivery and ROB/PRF probes use the timestamp counter (TSC). Their values are reported as TSC ticks, not measured core cycles. The CSV headings `IPC`, `Cycles`, and `median_tsc` do not change that distinction. Frequency, scheduling, SMT activity, and measurement overhead can affect cross-machine comparisons; within-machine ratios are more informative than absolute elapsed times alone.

Repeated sweeps. Skylark has three saved process-level rows at each filler count from 0 to 300 in increments of two. Artemisia has five at each count from 0 to 400 in increments of one. Each row contains a median and minimum/maximum from an internal trial loop. The source and runner default to 11 measured trials after one warmup, 20,000 iterations per trial, and a 256 MiB buffer; the actual historical command lines and internal trial counts were not recorded in the CSVs. The plots show the median of the saved process medians and the range of those medians. Shaded ranges are not confidence intervals or the full internal-trial range.

2 Exercise 1: Evidence of pipelined execution (10 points)

Method. Execute 20 integer adds per iteration for 200 million iterations, giving four billion explicit adds in each case. In the independent version, adds rotate among four registers; in the dependent version, every add updates `eax`. The representative assembly below is repeated to construct the 20-add bodies in `benchmark.cpp` from `pipline_test`.

```
add $1, %eax
add $1, %ebx
add $1, %ecx
add $1, %edx

add $1, %eax
add $1, %eax
```

Measurement	Skylark	Artemisia
Independent (s)	0.994961	0.465007
Dependent (s)	2.687604	1.487602
Dependent / independent	2.701	3.199

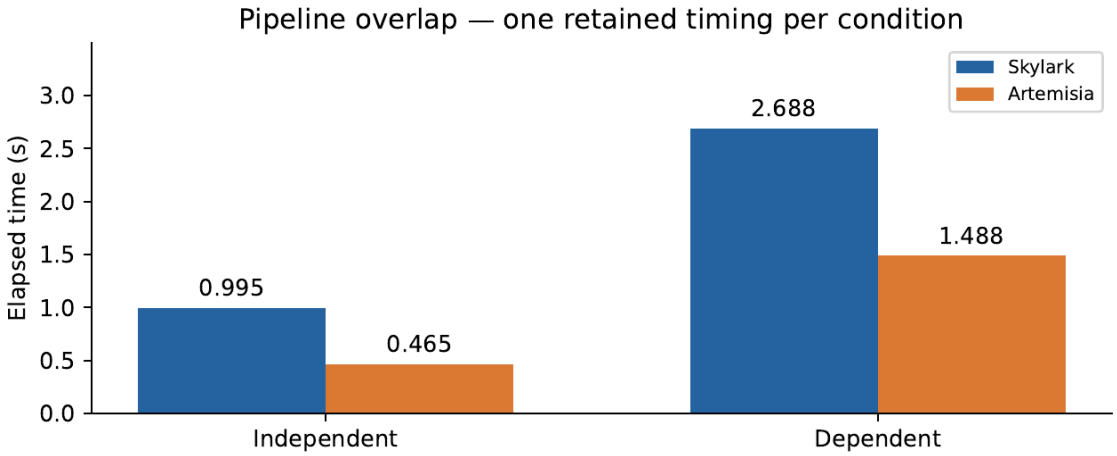


Figure 1: Pipeline probe: one retained elapsed-time measurement for each condition on each machine.

Finding. Independent updates are faster on both machines. The larger relative improvement on Artemisia is consistent with greater usable instruction-level parallelism for this workload. Dependencies restrict overlap; removing some dependencies improves throughput.

Interpretation limit. This comparison supports overlapped execution but does not isolate pipelining from multiple execution units or superscalar issue. It does not determine pipeline depth. The source does not explicitly initialize the assembly registers or declare a condition-code clobber, and the cases run in a fixed order. A stronger confirmation would sweep the number of independent recurrence chains using an operation with a measurable dependent latency, initialize operands, inspect the generated assembly, and repeat the timings.

3 Exercise 2: Evidence of out-of-order execution (10 points)

Method. Compare two instruction orders with the same two register dependency chains. Each loop contains five copies of a four-instruction group and executes 100 million iterations. The measured quantity is total elapsed time. The source names the immediate multiply/add pairing “poorly scheduled” and the multiply-first arrangement “manually scheduled.”

```
imul %eax, %eax
add  $1, %eax
imul %ebx, %ebx
add  $1, %ebx

imul %eax, %eax
imul %ebx, %ebx
add  $1, %eax
add  $1, %ebx
```

Measurement	Skylark	Artemisia
Immediate pairs (s)	1.343127	0.743780
Manually scheduled (s)	1.344682	0.743828
Immediate / manual	0.998844	0.999935

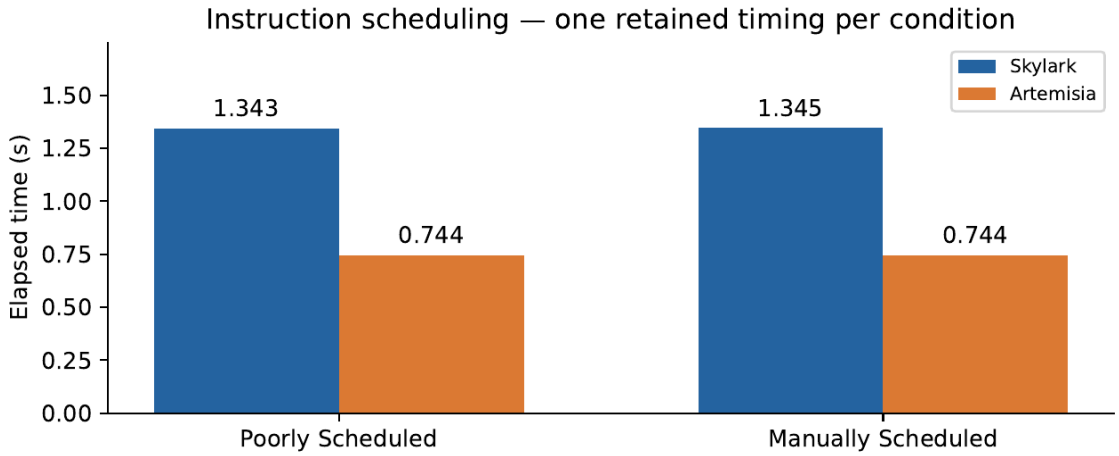


Figure 2: Scheduling comparison. The near-equal bars are individual measurements, not means over trials.

Finding. Reordering changes the recorded time by approximately 0.116% on Skylark and 0.006% on Artemisia, relative to the manual case. This is consistent with hardware scheduling recovering overlap between independent register chains.

Interpretation limit. Timing similarity alone is not a conclusive test of out-of-order execution. Both variants could be limited by the same recurrence throughput, and there is no serial control or sample distribution with which to judge these small differences. The source's two-chain test therefore provides suggestive evidence. A stronger test would compare long grouped dependency chains with interleaved chains and a serial baseline, while verifying the exact emitted instruction order. Registers and condition-code clobbers should also be made explicit before new measurements.

4 Exercise 3: Evidence of branch prediction (10 points)

Method. Generate 32,768 integers in the range 0–255. Scan the array 10,000 times, conditionally updating an accumulator when a value is at least 128. Repeat after sorting the same values, with sorting outside the timed region. This preserves the value distribution while making runs of branch outcomes more regular. Each timing covers 327,680,000 element visits.

```
for (int iter = 0; iter < 10000; ++iter)
    for (int i = 0; i < 32768; ++i)
        if (data[i] >= 128)
            sum ^= data[i];
// Time this loop; sort outside timing; reset sum; time again.
```

Measurement	Skylark	Artemisia
Unsorted (s)	4.849347	1.822171
Sorted (s)	1.851148	0.842667
Unsorted / sorted	2.620	2.162

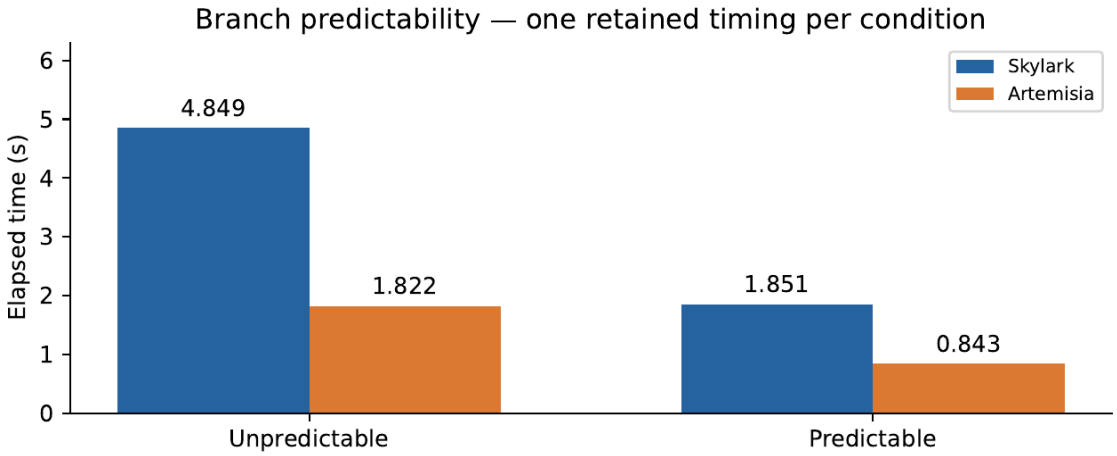


Figure 3: Sorted versus unsorted data, with one retained timing per condition.

Finding. Sorting reduces the recorded runtime on both machines; the relative penalty for unsorted data is greater on Skylark. If the executed kernel contains the intended conditional branch, this agrees with a predictor benefiting from more regular outcomes.

Interpretation limit. The source never observes the final accumulator. A compiler may eliminate the work or replace the conditional with branchless instructions; XOR does not prevent these transformations. Historical compiler flags and disassembly are absent, so these timings do not independently establish branch prediction. Retaining a checksum, verifying the branch in disassembly, adding matched branchless controls, and recording branch misses would strengthen the test. These measurements do not identify a neural, counter-based, or hybrid predictor.

5 Exercise 4: ROB size (30 points)

Method. Use the NOP control from `physicalreg.cpp`. Two pointer cursors traverse a randomized cycle, initially half a cycle apart. The loads are independent of each other but each cursor depends on its preceding load. Insert *N* NOPs after each load. A long-latency older load delays retirement, allowing younger instructions to accumulate. If a later independent load cannot enter the available window, overlap is reduced and time per iteration should rise.

```
mov (%0), %0          // advance pointer chain 1
.rept N
nop
.endr
mov (%1), %1          // advance pointer chain 2
.rept N
nop
.endr
```

Both gaps contain fillers, avoiding a short gap from the second load to the next iteration's first load. A requested huge-page allocation reduces translation pressure when honored, but `MADV_HUGEPAGE` does not prove that huge pages were allocated. Memory latency and other resources remain potential confounders.

Analysis. Reproduce the supplied runner's heuristic: take the medians of the first and last 10% of sweep points as low and high reference levels. If the high level is below 1.3 times the low level, report no clear step. Otherwise

locate the first crossing of 50% of the rise. The 10–90% crossings describe transition width, not statistical confidence.

c	Skylark	Artemisia
Sweep / launches per point	0–300, step 2 / 3	0–1000, step 1
NOP 50% crossing	$N \approx 222$	N ≈ 510
NOP 10–90% crossings	128–226	Not resolved
Candidate ROB capacity	Approximately 224 entries	Around 512

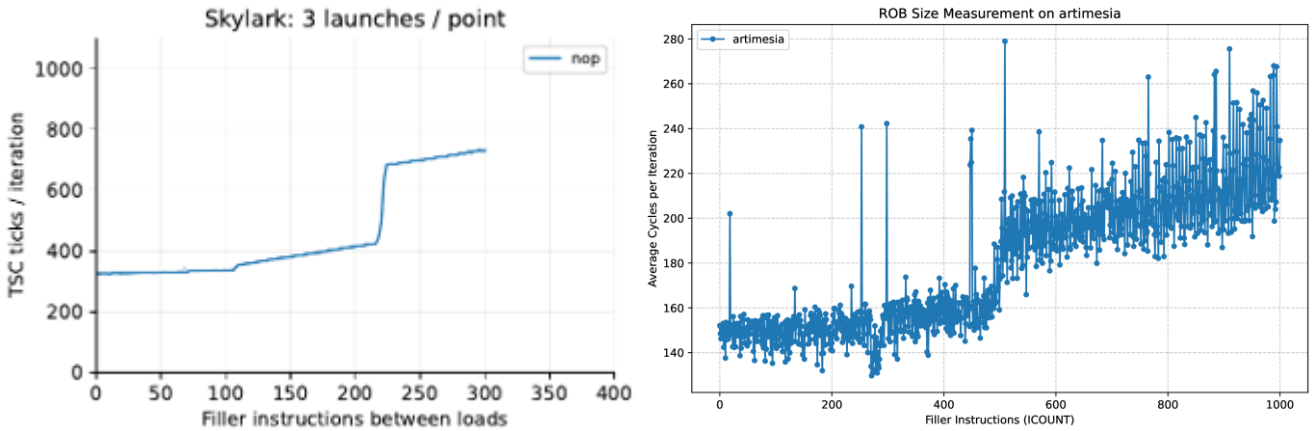


Figure 4: NOP control on both machines. Lines are medians of process medians; shading is their minimum-to-maximum range. Identical axis limits support comparison.

Finding. Skylark's heuristic transition is consistent with a window of roughly 222 + 2 = 224 instructions under the simplified two-load occupancy model. The broad transition and other loop instructions make this a tentative effective-window estimate, not an exact entry count. Artemisia shows around a 512 rob size and this is determined by the sharp vertical point near 512 and the cycles per iteration stayed stead after more filler instruction was added.

The separate original `rob_size/src/benchmark.cpp` inserts fillers only between the two loads, uses a fixed-stride list, and lacks a warmup. Its two saved PDF plots also both carry Artemisia labels, despite one filename mentioning Skylark. These ambiguous legacy plots are not used for the numerical comparison above; the repeated NOP control provides the traceable two-machine evidence.

6 Exercise 5: Physical register file size (35 points)

Method and register classes. Keep the same two-load probe but replace NOPs with instructions that allocate destination registers. Integer `add` updates 64-bit general-purpose registers; SSE `paddd` updates 128-bit SIMD registers. Rotate across eight destinations to limit recurrence pressure. Compare each curve against the NOP control: an earlier rise is consistent with register-allocation pressure occurring before the ROB control's rise.

```
add $1, %r8
add $1, %r9          // ... through r15

paddd %xmm0, %xmm8
```

```
paddd %xmm0, %xmm9   // ... through xmm15
```

50% crossing (10–90% range)	Skylark	Artemisia
NOP control	222 (128–226)	510
Integer ADD	110 (80–226)	320(198-396)
SSE SIMD	146 (96–230)	345 (240–790)
Integer LEA	No supplied measurement	No clear step through 400
AVX2 SIMD	No supplied measurement	203 (162–203)

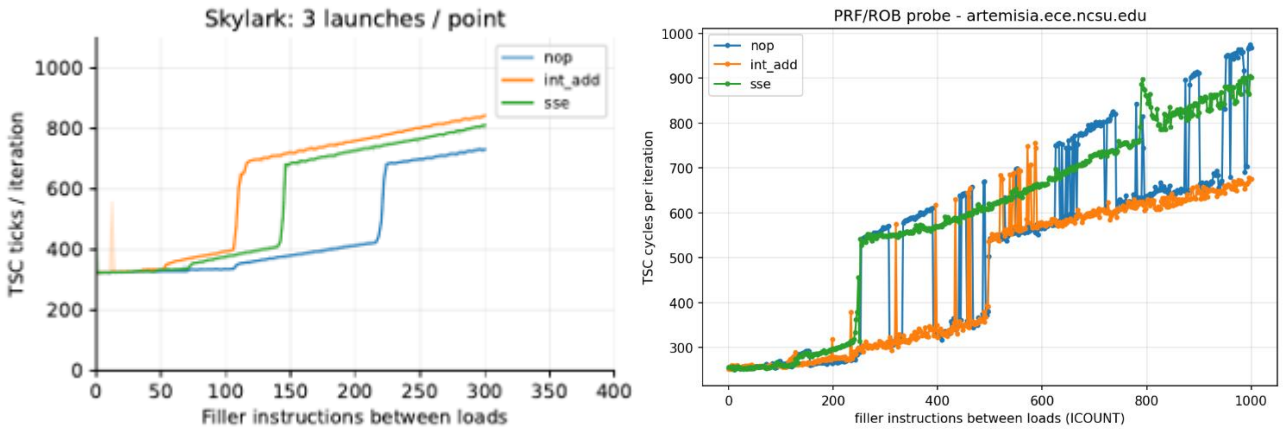


Figure 5: Repeated ROB/PRF probes for the register classes measured on both machines. Shading shows the range of saved process medians. LEA and AVX2 are supplementary Artemisia-only observations.

Finding: Skylark's integer ADD and SSE curves cross their thresholds before the NOP control, at approximately 110 and 146 fillers, respectively. These are candidate effective speculative allocation budgets. Artemisia's extended sweep produces automated 50% crossings of 254 for NOP, 320 for integer ADD, and 354 for SSE. However, its curves contain spikes and substantial upward trends, so these values do not consistently identify the main visible steps. The graph shows an SSE transition around 248–252 fillers and a sustained integer ADD transition near 498–500 fillers. The NOP curve also has a sustained rise near 500 fillers, alongside earlier excursions to higher timings. Thus, Artemisia exhibits a larger observed SSE allocation window than Skylark under this probe, while its integer ADD result does not clearly distinguish an integer-register limit from the control's limiting resource.

Interpretation limit: Extending the sweep changes the endpoint timing levels used by the automated crossing calculation. Consequently, Artemisia's updated SSE crossing of 354 should not be interpreted as a physical allocation threshold increasing from the previously reported 247. Adding two pointer loads to a crossing is likewise a window-occupancy convention, not a universal conversion to PRF capacity: those loads allocate integer destinations, not SIMD destinations. Non-speculative mappings, other allocation overhead, flag writes from ADD, scheduler limits, and execution throughput can also influence the measurements. These results therefore characterize observed thresholds under the probe rather than exact total PRF capacities.

The earlier Artemisia SSE and AVX2 crossings, 247 and 203, were obtained with different instruction encodings and vector widths and do not establish separate physical register files. The supplied extended sweep

does not include AVX2 or integer LEA, so their earlier results remain supplementary. No corresponding Skylark AVX2 measurement was supplied; its absence does not establish unsupported hardware.

7 Exercise 14: Effective instruction delivery (10 points)

Method. Run one million iterations of a loop containing *N* NOP instructions, with bodies of 10, 50, 100, 200, 400, 600, or 800 NOPs. The source expands the NOP body at compile time. It records a TSC interval and divides the explicit NOP count by that interval.

```
start = rdtsc();
for (int i = 0; i < 1000000; ++i) {
    __asm__ __volatile__(NOP400); // example: 400 NOPs
}
end = rdtsc();
double rate = (400.0 * 1000000) / (end - start);
```

NOPs per iteration	Skylark (NOPs/tick)	Artemisia (NOPs/tick)
10	0.882150	4.639346
50	2.930842	6.724090
100	3.016185	7.066403
200	3.059096	7.232266
400	3.083490	7.429315
600	3.091218	7.571737
800	3.095427	7.606324

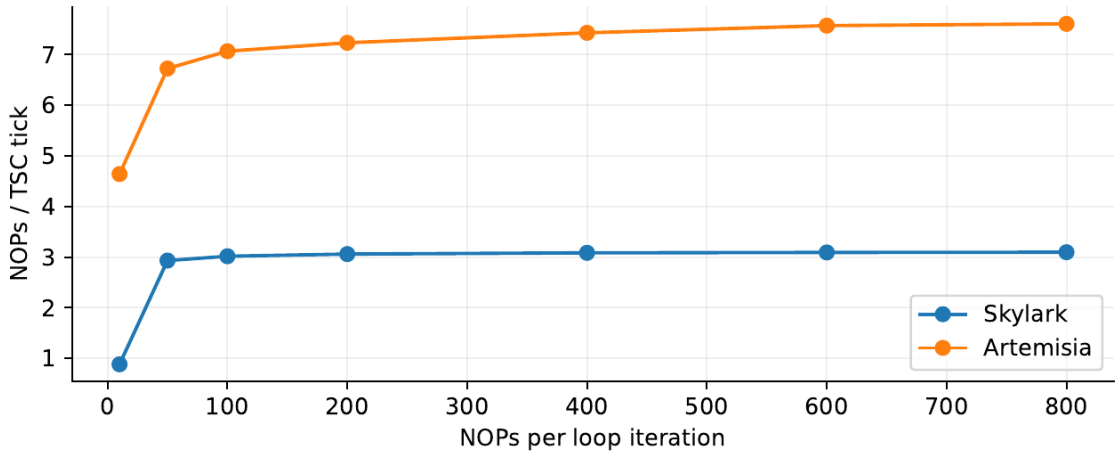


Figure 6: Effective NOP throughput. Each point is one saved measurement; the CSV's IPC field is relabeled to reflect the timer actually used.

Finding. Throughput approaches approximately 3.08–3.10 NOPs/TSC tick on Skylark and 7.43–7.61 on Artemisia for the 400–800-NOP bodies. Longer bodies amortize loop overhead, and Artemisia delivers more NOP work per TSC tick in these measurements.

Interpretation limit. These are effective throughput estimates, not isolated instruction-fetch widths. The TSC is not a core-cycle counter; the code does not serialize `RDTSC`; and small hot loops may be supplied by decoded-instruction structures. Decode, retirement, frequency, alignment, and loop overhead may all influence the result.

Rounding the measured maximum up to 4 or 8 would not establish a hardware fetch limit. Measured core cycles and repeated samples are needed to answer the per-core-cycle question quantitatively.

8 Enclave status and remaining evidence

Enclave experiments were not performed. There are no inside-enclave results or paired enclave/non-enclave comparisons in this report. No claim is made that SGX or enclave access was unavailable: that reason was not established. The assignment requests enclave comparison or a documented support/access limitation, so this component remains unfulfilled unless an applicable limitation is documented.

Before submission, complete Skylark's CPU identification and collect repeated timings for Exercises 1, 2, 3, and 14 if those requirements are to be met. Preserve the per-run samples, build flags, affinity, and disassembly. The branch experiment needs observable output and verification that its conditional branch remains. Exercise 4 on Artemisia and exact PRF capacities remain unresolved by the available sweeps. These are limitations of the evidence, not values to replace with published specifications.

9 Data and reproduction

The accompanying project contains the exact CSV inputs in `data/`, benchmark sources in `source/`, and `build_figures.py`. Run the following from the report directory to regenerate the figures and PDF without running benchmarks:

```
python3 build_figures.py   # requires matplotlib
pdflatex main.tex
pdflatex main.tex
```

Upload the complete ZIP as an Overleaf project and select `main.tex` with pdfLaTeX. The figures are included as PDFs, so Python is unnecessary on Overleaf. The accompanying README describes how to preserve new measurements separately. This report uses the source files actually supplied; the repository's top-level README describes revised harness files that were absent from the inspected checkout and is not evidence that those revisions generated these results.

References

[1] ECE 492/592, Homework 3, Fall 2026, supplied assignment PDF, pp. 2–4.

[2] Intel, *Intel Xeon Gold 5420+ Processor: Specifications*. Product code name used only for machine identification; no ROB, PRF, or fetch-width specification is substituted for an experimental finding. https://www.intel.com/content/www/us/en/products/sku/232381/intel-xeon-gold-5420-processor-52-5m-cache-2-00-ghz/specifications.html

AI citations

Assistance provided:

- Read the assignment, benchmark sources, configuration snapshot, and save data.
- Confirmed file-to-machine mapping with the user.
- Checked the published Sapphire Rapids identification on Intel's product page.
- Generated side-by-side plots from existing CSVs and checked the sweep thresholds.
- Drafted main.tex, including methodology, snippets, results, and evidence limits.
- Compiled the report and packaged the Overleaf project.
- Help code the pipeline, branch predictor, rob size, physical reg, OoOE test, and instruction with.
- Also wrote the python scripts.

Used Codex and Gemini for research and coding.